Efficient Error and Variance Estimation for Low-Rank Matrix Approximation

Byungwoo Lee* Yifan Yang† Adrian Zhou‡

Abstract. Our study focused on efficient diagnostics for low-rank approximations such as randomized SVD. Specifically, we analyze a leave-one-out (LOO) estimator for approximation error and a matrix Jackknife estimator for variance. We show that LOO is unbiased and computationally efficient, while the Jackknife provides a conservative yet stable variance estimate. Numerical experiments confirm their accuracy and practicality for large-scale randomized linear algebra.

1 Introduction

Working with huge matrices for real-world applications is computationally demanding, with standard methods like SVD being too slow and memory-intensive. Faster, randomized techniques provide good approximations but introduce uncertainty. Therefore, we need efficient and inexpensive diagnostic tools to check the quality of these randomized results, preferably by using information the main algorithm has already computed. Epperly and Tropp[2] proposed two computationally efficient tools:

- A **Leave-one-out error** estimator $||A - X||_F^2$, which tells us roughly how much approximation error to expect between matrix A and its low-rank estimate X.
- A **Jackknife Variance** estimator $\mathbb{E}\left[(X - \mathbb{E}[X])^2\right]$, which checks stability by measuring how much approximations change when you drop each sampled column.

1.1 Related Work

In this section, we present an overview of commonly used methods for estimating the error and variance associated with randomized singular value decomposition (RSVD) algorithms.

*UC Davis, bywlee@ucdavis.edu
†UC Davis, yfnyang@ucdavis.edu
‡UC Davis, chnzhou@ucdavis.edu

1.1.1 Error Estimation

Direct Calculation The most direct way to measure the error of a matrix approximation is to simply calculate it. For instance, with the randomized SVD, the squared error can be found quickly using the formula $||A - X||_F^2 = ||A||_F^2 - ||X||_F^2$, which is proved in the Appendix. This direct method works well when the entire matrix A is stored in memory. However, it is often not possible or is too expensive in other cases. For example, in "black-box" situations, you can't see the whole matrix and can only interact with it by multiplying it with vectors. It also fails when data streams in piece by piece. In these scenarios, calculating the error of the original matrix A directly is too hard or even impossible.

GH Estimator The Girard-Hutchinson (GH) estimator is a popular method for estimating the error $||A - X||_F^2$ of a matrix approximation. It works by multiplying the error matrix, $A - X$, with m random test vectors (ν_i) and averaging the results, based on the formula $\widehat{\mathrm{Err}}_{GH}^2 = \frac{1}{m}\sum_{i=1}^{m} ||(A - X)\nu_i||^2$. The accuracy of this estimate improves as we use more test vectors (t). The GH estimator is a good benchmark because it only needs matrix-vector products, just like the randomized algorithms it helps to check. However, its main disadvantage is the computational cost. It requires m *extra* matrix-vector multiplications with A. Since these multiplications are usually the most time-consuming part, a large m can make the error check very slow, potentially wiping out the efficiency gained from using a randomized method in the first place.

1.1.2 Variance Estimation

Beyond error estimation, understanding the variability or stability of a randomized algorithm's output is crucial.

Tukey's Jackknife Estimator Tukey's jackknife is a classic method for estimating the variance of a statistic. The core idea is simple: if your statistic is calculated from a set of s data samples, you recalculate it s times, each time **leaving one sample out**. The variance is then estimated by measuring how much these "leave-one-out" results ($E^{(j)}$) differ from their average ($E^{(\cdot)}$), using the formula:

$$\widehat{\mathrm{Var}}(E_{s-1}) := \sum_{j=1}^{s} |E^{(j)} - E^{(\cdot)}|^2.$$

This process reveals how sensitive the statistic is to individual data points. The theory behind this, **Efron-Stein-Steele inequality**, shows an important property: on average, the jackknife estimator tends to **overestimate** the true variance. This means it provides a slightly conservative, or cautious, estimate of the variance.

Bootstrap Resampling Bootstrap resampling is a method to check the accuracy of an estimate. The core idea is to create new datasets from your original one by **sampling with replacement**. This means you repeatedly pick samples from the original data to form many new "bootstrap" datasets, each the same size as the original, where the same data point can be chosen multiple times. The process follows three main steps:

1. **Resample:** Create B new bootstrap datasets by sampling your original data (for example, the columns of a matrix Ω) with replacement.

2. **Re-compute:** Run your algorithm on each new bootstrap dataset to get a new result or statistic $S^{(j)}$.

3. **Estimate Variance:** The bootstrap's variance estimate is simply the sample variance of all the new results just calculated. This shows how much the original statistic is expected to vary. The formula is:

$$\widehat{\mathrm{Var}}_{\mathrm{boot}}(S) = \frac{1}{B-1}\sum_{j=1}^{B}(S^{(j)} - \bar{S})^2,$$

   where $\bar{S} = \frac{1}{B}\sum_i^B S^{(i)}$ is the average of the new results.

2 Methodology

Randomized matrix algorithms have gained wide application in scientific computing and machine learning due to their scalability and efficiency. However, ensuring their reliability requires robust diagnostic tools. This section introduces two key techniques—Leave-One-Out (LOO) error estimation and the Jackknife variance estimation—for assessing the quality and variability of randomized low-rank approximations such as the randomized SVD.

2.1 Leave-One-Out (LOO) Error Estimation

The leave-one-out error estimator is a tool to check the quality of an approximation (X) for a matrix (A) that was created by a randomized algorithm. The basic idea is to see how the approximation changes when you temporarily remove each of the random vectors (ω_j) that was used to create it.
We then estimate the error by averaging how well each of these slightly different approximations performs on the specific vector that was left out, using the formula:

$$\widehat{\mathrm{Err}}^2(X_{s-1}, A) := \frac{1}{s}\sum_{j=1}^{s}||(A - X^{(j)})\omega_j||^2$$

A key feature is that this method gives an **unbiased estimate** of the true error, meaning it's correct on average. The main benefit is its **efficiency**; it reuses information the algorithm already has, avoiding extra, costly steps. This helps you know how reliable your result is and choose good settings for the algorithm.

2.2 Jackknife Variance Estimation

Instead of estimating error, the jackknife method estimates **variance**. This measures how sensitive or unstable the algorithm's output (X) is to its specific random inputs (ω_j). The method works by creating several "leave-one-out" versions of the output ($X^{(j)}$) and measuring how much they differ from their average ($X^{(\cdot)}$). The formula for the jackknife variance estimate is:

$$\text{Jack}^2(X_{s-1}) := \sum_{j=1}^{s} ||X^{(j)} - X^{(\cdot)}||_F^2$$

A high variance suggests the result is unstable, which can mean the problem is difficult or more samples are needed. A key property is that the jackknife tends to **overestimate** the true variance, giving a conservative, or cautious, result. This information helps users choose better algorithm settings and warns them about potential instability.

2.3 Comparison with Prior Methods

Epperly and Tropp's[2] leave-one-out methods are presented as more efficient and better-integrated tools for checking the quality of randomized matrix algorithms compared to previous diagnostics.

Comparison with GH Estimator The LOO estimator is much **cheaper** than the standard GH estimator.

- **No Extra Cost:** The main advantage is that the LOO estimator reuses information the main algorithm has already computed. It **does not require any extra, expensive multiplications** with the full matrix A. The GH method, in contrast, requires these extra steps, which can be slow and costly.

- **Automatic Accuracy Improvement:** The LOO estimate **naturally gets more accurate** as the main approximation gets better (for example, as its rank s increases). To improve the GH estimate, you have to separately increase its own parameter m, which adds more computation cost.

Because the LOO estimator is so tightly connected to the main algorithm's parameters, it's a more efficient and clever tool for estimating error.

Comparison with Bootstrap and Matrix Jackknife Standard bootstrap methods don't work well for advanced algorithms like the randomized SVD. The reason is that bootstrap samples data *with replacement*, which creates test matrices with duplicate columns and fewer *unique* ones. The quality of the randomized SVD depends heavily on the number of unique columns, so the bootstrap versions produce lower-quality estimation. This makes their results vary wildly, causing **the bootstrap estimation to be unstable**, which you can observe in the simulation part. This is why Epperly and Tropp developed their **matrix jackknife**. While standard bootstrap may work for simpler "Monte Carlo" style algorithms, those methods are not very accurate. The matrix jackknife is specifically designed for the powerful and widely-used algorithms like randomized SVD and Nyström. These advanced methods are far more accurate, and their error behaves differently. Therefore, you need a specialized tool like the matrix jackknife to correctly analyze these better algorithms, as a tool designed for simpler methods isn't reliable.

3 Theoretical Results

Theorem 1 (Leave-one-out error estimator)**.** *Let $\omega_1, \ldots, \omega_s$ be independent and identically distributed (iid) random vectors in $\mathbb{K}^{d_2}$ that are isotrophic, i.e. $\mathbb{E}[\omega_j \omega_j^*] = I_{d_2}$. Let X denote one of two matrix estimators defined for s or $s-1$ inputs: $X : (\mathbb{K}^{d_2})^s \to \mathbb{K}^{d_1 \times d_2}$ or $X : (\mathbb{K}^{d_2})^{s-1} \to \mathbb{K}^{d_1 \times d_2}$. Define estimates $X_s := X(\omega_1, \ldots, \omega_s)$ and $X_{s-1} := X(\omega_1, \ldots, \omega_{s-1})$. Define replicates $X^{(j)} = X(\omega_1, \ldots, \omega_{j-1}, \omega_{j+1}, \ldots, \omega_s)$. Define leave-one-out mean-square error estimate by $\widehat{\mathrm{Err}}^2(X_{s-1}, A) := \frac{1}{s} \sum_{j=1}^{s} \|(A - X^{(j)})\omega_j\|^2$.. Then,*

$$\mathbb{E}\widehat{\mathrm{Err}}^2(X_{s-1}, A) = \mathbb{E}\|A - X^{(j)}\|_F^2.$$

Proof. Note that $X^{(j)}$ is independent for ω_j.

$$\begin{aligned}
\mathbb{E}_{\omega_j} \left\|(A - X^{(j)})\omega_j\right\|^2 &= \mathbb{E}_{\omega_j} \left[\omega_j^* (A - X^{(j)})^* (A - X^{(j)})\, \omega_j\right] \\
&= \mathbb{E}_{\omega_j} \operatorname{tr}\left[(A - X^{(j)})^* (A - X^{(j)})\, \omega_j\, \omega_j^*\right] \\
&= \operatorname{tr}\Big((A - X^{(j)})^* (A - X^{(j)})\, \mathbb{E}_{\omega_j}[\, \omega_j\, \omega_j^*\,]\Big) \\
&= \operatorname{tr}\big((A - X^{(j)})^* (A - X^{(j)})\big) = \|A - X^{(j)}\|_F^2.
\end{aligned}$$

The first and the last line is a property of Frobenius norm, the second line is the cyclic property of the trace, the third line is from the independence of $X^{(j)}$ and ω_j, the fourth line is from the isotropic assumption of ω_j.

By the tower property,

$$\begin{aligned}
\mathbb{E}\,\widehat{\mathrm{Err}}^2(X_{s-1}, A) &= \frac{1}{s}\sum_{j=1}^{s}\mathbb{E}_{-\omega_j}\left[\mathbb{E}_{\omega_j}\left\|(A - X^{(j)})\omega_j\right\|^2\right] \\
&= \frac{1}{s}\sum_{j=1}^{s}\mathbb{E}_{-\omega_j}\left\|A - X^{(j)}\right\|_{\mathrm{F}}^2 \\
&= \frac{1}{s}\sum_{j=1}^{s}\mathrm{MSE}(X_{s-1}, A) = \mathrm{MSE}(X_{s-1}, A).
\end{aligned}$$

where $\mathbb{E}_{-\omega_j}$ denotes $\mathbb{E}_{\omega_1,\ldots,\omega_{j-1},\omega_{j+1},\ldots,\omega_s}$. The first line is the definition of the estimator, the second line is what we have shown above, the third line is the definition of MSE. □

Theorem 1 states that our leave-one-out mean-square error estimator is unbiased for $\mathrm{MSE}(X_{s-1}, A)$, thereby justifying its use. However, it does not guarantee the robustness of the estimator, which motivates the use of an additional diagnostic: the jackknife variance estimator.

In the proof of Theorem 1, we used the assumptions of independence and isotropy, both of which are necessary. For the well-definedness of X_s and X_{s-1}, the assumption of identical distribution is also required. Overall, all assumptions are essential.

Numerically, the vectors ω_j were generated as i.i.d. Gaussian samples for convenience. In fact, it suffices for them to be isotropic, so the Gaussian assumption is stronger than necessary. We can also verified that other test vectors are acceptable and actually perform well (Figure 4).

Proposition 2 (Efron–Stein–Steele inequality)**.** *Let $\omega_1, \ldots, \omega_s \in \Omega$ be independent elements in a measurable space Ω, and let $f : \Omega^s \to \mathbb{K}$ be measurable. Let $(\omega'_j : j = 1, \ldots, s)$ be an independent copy of $(\omega_j : j = 1, \ldots, s)$. Then*

$$\mathrm{Var}\left(f(\omega_1, \ldots, \omega_s)\right) \leq \frac{1}{2}\sum_{i=1}^{s}\mathbb{E}\left|f(\omega_1, \ldots, \omega_s) - f(\omega_1, \ldots, \omega_{i-1}, \omega'_i, \omega_{i+1}, \ldots, \omega_s)\right|^2.$$

Proof. Using telescoping argument and basis expansion of Hilbert space. [1], [5]. Details in Appendix A. □

The complex-valued version of the inequality presented here follows from the more standard version for real values by treating the real and imaginary parts separately.

Proposition 2 was originally intended to deal with Tukey's jackknife variance estimator, which is the jackknife estimator for scalar. In our paper, we extend it from scalar($\mathbb{K}$) to matrix($\mathbb{K}^{d_1 \times d_2}$), as following Theorem 3.

Theorem 3 (Matrix jackknife)**.** *Let $\omega_1, \ldots, \omega_s$ be independent and identically distributed (iid) random vectors in a measurable space Ω. Let X denote one of two matrix estimators*

defined for s or $s-1$ inputs: $X : \Omega^s \to \mathbb{K}^{d_1 \times d_2}$ or $X : \Omega^{s-1} \to \mathbb{K}^{d_1 \times d_2}$. Assume X is invariant to reordering of its inputs: for any permutation π, $X(\omega_1, \ldots, \omega_s) = X(\omega_{\pi(1)}, \ldots, \omega_{\pi(s)})$. Define estimates $X_s := X(\omega_1, \ldots, \omega_s)$ and $X_{s-1} := X(\omega_1, \ldots, \omega_{s-1})$. Define jackknife replicates $X^{(j)}$ and their average $X^{(\cdot)}$ by $X^{(j)} = X(\omega_1, \ldots, \omega_{j-1}, \omega_{j+1}, \ldots, \omega_s), X^{(\cdot)} = \frac{1}{s}\sum_{j=1}^{s} X^{(j)}$. Define the matrix jackknife estimate $\mathrm{Jack}^2(X_{s-1}) := \sum_{j=1}^{s} \|X^{(j)} - X^{(\cdot)}\|_F^2$.. Then,

$$\mathrm{Var}(X_{s-1}) \leq \mathbb{E}\,\mathrm{Jack}^2(X_{s-1}).$$

Proof. Fix a pair of indices $1 \leq m \leq d_1$ and $1 \leq n \leq d_2$. Since the samples $\omega_1, \ldots, \omega_{s-1}$ are identically distributed and X invariant to reordering, the last sample ω_s can be used to fill the role of each ω'_j. For the (m, n)-matrix entry $(X_{s-1})_{mn}$, we observe

$$\begin{aligned}
\mathrm{Var}((X_{s-1})_{mn}) &\leq \frac{1}{2}\sum_{i=1}^{s-1} \mathbb{E}\left((X^{(s)})_{mn} - (X^{(i)})_{mn}\right)^2 \\
&= \frac{1}{2s}\sum_{i,j=1}^{s} \mathbb{E}\left((X^{(i)})_{mn} - (X^{(j)})_{mn}\right)^2 \\
&= \sum_{j=1}^{s} \mathbb{E}\left((X^{(j)})_{mn} - (X^{(\cdot)})_{mn}\right)^2.
\end{aligned}$$

The first line follows from Proposition 2; the second line uses the fact that all $X^{(i)}$ are identically distributed; and the third line is obtained by expanding the square and rearranging terms. Summing this equation over all $1 \leq m \leq d_1$ and $1 \leq n \leq d_2$ yields the stated result. □

Theorem 3 states that our estimator overestimates the variance of X_{s-1}, which means we can find an upper bound of the variance. So when the jackknife estimate is small, we can conclude the variance is also small, implying that the original estimate is pretty reliable. Moreover, we can use this estimate for choosing appropriate s. If we find "good" s having sufficiently small jackknife variance estimate, then the original estimate using such s would be reliable.

Assumptions of the theorem are not strong that are natural and reasonable. First, it generalized the space of ω_is from $\mathbb{K}^{d_2}$ to Ω, which is weak and nice assumption from Proposition 2. All the other assumptions are necessary since they are needed to use Proposition 2. Same as Theorem 1, the assumptions are reasonable practically.

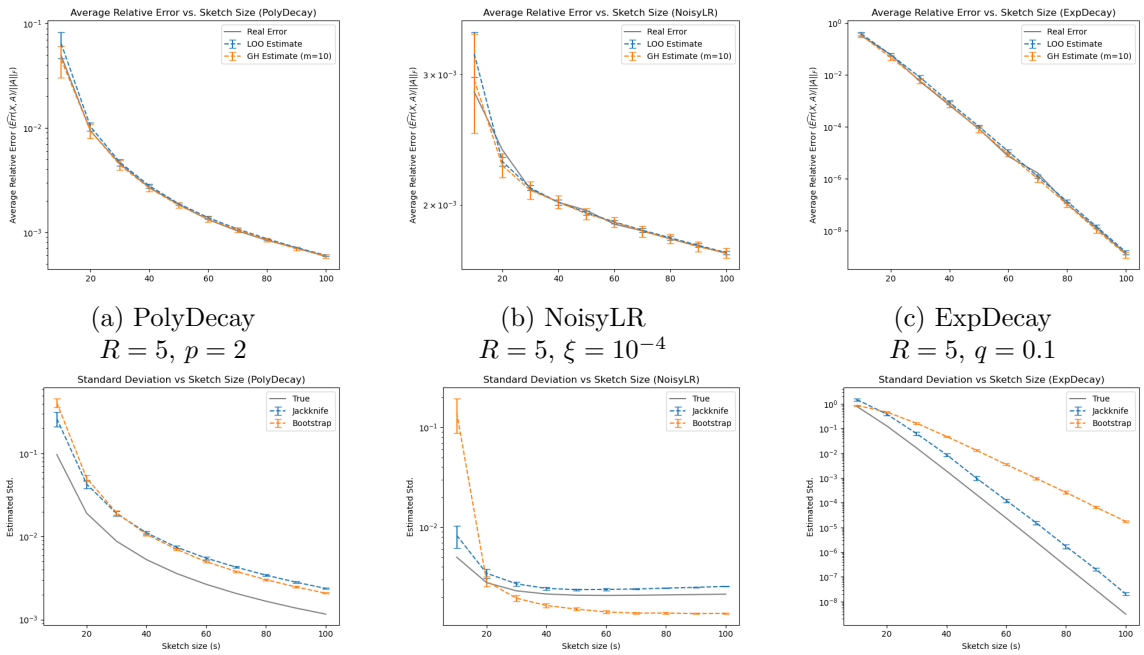
To resolve the biasness of jackknife variance estimator, the author suggested a future work related to a natural analog of Quenouille's scalar jackknife bias estimate.[4]

4 Simulation Design and Results

4.1 Setup

Randomized SVD[3] is one of the most useful low dimension approximation method. The specific algorithm is represented in Algorithm 1. Let's consider 3 different kinds of low-rank matrics: *PolyDecay*, *NoisyLR*, and *ExpDecay* [6] in Algorithm B. We will use different methods for Error and Variance estimation. *(Codes for simulation are in the* **??***.)*

4.2 Results



(a) PolyDecay $R = 5$, $p = 2$

(b) NoisyLR $R = 5$, $\xi = 10^{-4}$

(c) ExpDecay $R = 5$, $q = 0.1$

(d) PolyDecay $R = 5$, $p = 2$

(e) NoisyLR $R = 5$, $\xi = 10^{-4}$

(f) ExpDecay $R = 5$, $q = 0.1$

Figure 1: Different methods for Error and Variance estimation for randomized SVD

Both LOO and Girard-Hutchinson yield unbiased error estimates (Figure 1abc), but LOO is faster and exhibits lower variance over repeated trials (Figure 2def). The Jackknife estimator $\mathrm{Jack}(X)$ provides a stable and conservative variance estimate, while the Bootstrap estimator $\mathrm{Boot}(X)$ is highly sensitive to the choice of A (Figure 1def), making it unsuitable for RSVD. In addition, using uniformly distributed random matrix Ω can achieve similar accuracy with lower runtime (Figure 4). *More discussion will be covered in the Appendix B.*

References

[1] B. Efron and C. Stein. "The Jackknife Estimate of Variance". In: *The Annals of Statistics* 9.3 (1981), pp. 586–596. ISSN: 00905364, 21688966. URL: http://www.jstor.org/stable/2240822.

[2] Ethan N. Epperly and Joel A. Tropp. "Efficient error and variance estimation for randomized matrix computations". In: *SIAM Journal on Scientific Computing* 46.1 (2024), A508–A528. DOI: 10.48550/arXiv.2207.06342.

[3] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions". In: *SIAM Review* 53.2 (2011), pp. 217–288.

[4] M. H. Quenouille. "Approximate Tests of Correlation in Time-Series". In: *Journal of the Royal Statistical Society. Series B (Methodological)* 11.1 (1949), pp. 68–84. ISSN: 00359246. URL: http://www.jstor.org/stable/2983696 (visited on 06/07/2025).

[5] J. Michael Steele. "An Efron-Stein Inequality for Nonsymmetric Statistics". In: *The Annals of Statistics* 14.2 (1986), pp. 753–758. DOI: 10.1214/aos/1176349952. URL: https://doi.org/10.1214/aos/1176349952.

[6] Joel A. Tropp et al. "Fixed-Rank Approximation of a Positive-Semidefinite Matrix from Streaming Data". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. Curran Associates, Inc., 2017, pp. 1225–1234. URL: https://papers.nips.cc/paper/2017/hash/4558dbb6f6f8bb2e16d03b85bde76e2c-Abstract.html.

A Additional Proof Details

Fact. Let $A \in \mathbb{R}^{m\times n}$ and $Q \in \mathbb{R}^{m\times k}$ be a matrix with orthonormal columns (i.e., $Q^TQ = I$). Define $X = QQ^TA$. Then:

$$\|A - X\|_F^2 = \|A\|_F^2 - \|X\|_F^2.$$

Proof. We have

$$\begin{aligned}\|A\|_F^2 &= \|(A - X) + X\|_F^2 \\ &= \|A - X\|_F^2 + \|X\|_F^2 + 2\langle A - X, X\rangle_F\end{aligned}$$

Consider the cross term:

$$\begin{aligned}\langle A - X, X\rangle_F &= \langle A, X\rangle_F - \|X\|_F^2 \\ &= \mathrm{Tr}(A^TX) - \mathrm{Tr}(X^TX) \\ &= \mathrm{Tr}(A^TQQ^TA) - \mathrm{Tr}(A^TQQ^TQQ^TA) \quad (\text{since } X = QQ^TA \text{ and } Q^TQ = I) \\ &= \mathrm{Tr}(A^TQQ^TA) - \mathrm{Tr}(A^TQQ^TA) \\ &= 0\end{aligned}$$

Therefore, we conclude

$$\begin{aligned}\|A\|_F^2 &= \|A - X\|_F^2 + \|X\|_F^2 \\ \Rightarrow \quad \|A - X\|_F^2 &= \|A\|_F^2 - \|X\|_F^2\end{aligned}$$

□

Proof of Proposition 2 Let $Z = f(\omega_1, \ldots, \omega_s)$, $Z_j = f(\omega_1, \ldots, \omega_{j-1}, \omega'_j, \omega_{j+1}, \ldots, \omega_s)$, $Z'_0 := Z$, and $Z'_j = f(\omega'_1, \ldots, \omega'_{j-1}, \omega'_j, \omega_{j+1}, \ldots, \omega_s)$. Note that $Z'_j \overset{d}{=} Z$ for all $i = 0, \ldots, s$. Without loss of generality, we can assume $\mathbb{E}Z = 0$. Then, our claim is $\mathrm{Var}(Z) \leq \frac{1}{2}\sum_{i=1}^s \mathbb{E}(Z - Z_i)^2$.

$$\begin{aligned}\mathrm{Var}(Z) &= \frac{1}{2}\left(\mathbb{E}(Z - \mathbb{E}Z)^2 + \mathbb{E}(Z - \mathbb{E}Z)^2\right) \\ &= \frac{1}{2}\left(\mathbb{E}(Z'_0 - \mathbb{E}Z)^2 + \mathbb{E}(Z'_s - \mathbb{E}Z)^2\right) = \frac{1}{2}\mathbb{E}(Z'_0 - Z'_s)^2 \\ &= \frac{1}{2}\mathbb{E}\left[\sum_{i=0}^{s-1}(Z'_i - Z'_{i+1})\right]^2 \\ &\leq \frac{s}{2}\sum_{i=0}^{s-1}\mathbb{E}(Z'_i - Z'_{i+1})^2 = \frac{s}{2}\sum_{i=1}^{s}\mathbb{E}(Z - Z_i)^2.\end{aligned}$$

The third line is by telescoping sum, fifth line is by Cauchy-Schwartz inequality, last line is by the property $Z_i' - Z_{i+1}' \overset{d}{=} Z - Z_{i+1}$. This result means that if $\mathbb{E}Z^2{=}\infty$, then our claim holds. So it is enough to consider $\mathbb{E}Z^2 < \infty$ i.e. $Z \in (L^2(dF))^s$. Denote the orthonormal basis of the Hilbert space $L^2(dF)$ as $(\phi_k, k = 0, 1, \dots)$, $\phi_0 \equiv 1$. Let $\mathbf{k} = (k_1, \dots, k_s)$ and define $\phi_{\mathbf{k}}(\boldsymbol{\omega}) = \prod_{i=1}^s \phi_{k_i}(\omega_i)$. Then there exists constants $c(\mathbf{k})$ such that $f(\omega_1, \dots \omega_s) = \sum_{\mathbf{k}} c(\mathbf{k})\phi_{\mathbf{k}}(\boldsymbol{\omega})$. Then,

$$\mathbb{E}ZZ_i = \sum_{\mathbf{k},\mathbf{l}} c(\mathbf{k})c(\mathbf{l})\mathbb{E}\left[\phi_{k_i}(\omega_i)\phi_{l_i}(\omega_i')\prod_{j\neq i}\phi_{k_j}(\omega_j)\phi_{l_j}(\omega_j)\right] = \sum_{\mathbf{k}:k_i\neq 0} c(\mathbf{k})^2$$

Last equation is from orthogonality of ϕ_ks, independence of ω_js and ω_i', and the fact that only ϕ_0 is not centered among ϕ_ks.

$$\begin{aligned}
\frac{1}{2}\mathbb{E}\sum_{i=1}^s (Z - Z_i)^2 &= s\mathbb{E}Z^2 - \sum_{i=1}^s \mathbb{E}ZZ_i \\
&= s\mathbb{E}Z^2 - \sum_{\mathbf{k}} c^2(\mathbf{k})\#(k_i = 0) \\
&\geq s\mathbb{E}Z^2 - (s-1)\sum_{\mathbf{k}} c^2(\mathbf{k}) \\
&= s\mathbb{E}Z^2 - (s-1)\mathbb{E}Z^2 = \mathrm{Var}(Z)
\end{aligned}$$

The third line used the fact if $k_i = 0$ for all i, then $c(\mathbf{0}) = 0$ so that enough to think the number of nonzero k_i is at most $s - 1$, the fourth line is property of the basis.

B Simulation Experiment Results

Algorithm 1 Randomized SVD (Sketched parameter = s approximation)

Require: Matrix $A \in \mathbb{R}^{d_1 \times d_2}$, sketched parameter s, power iterations q
Ensure: Approximate SVD: $X(\Omega) := U_s \Sigma_s V_s^T \approx A$

```
Generate a Random Sketching Matrix Ω ∈ R^{d_2 × s}
Y ← AΩ
for i = 1 to q do
    Y ← A(A^T Y)                                  ▷ Optional power iteration
end for
Compute Economy QR decomposition: Y = QR
C ← Q^T A
Compute SVD: C = W Σ_s V_s^T
U_s ← QW
return U_s, Σ_s, V_s
```

In this simulation, we will consider the following 3 matrices that were introduced in (Tropp 2017[6]):

$$\text{(PolyDecay)} \quad \mathbf{A} = \operatorname{diag}(\underbrace{1, \ldots, 1}_{R \text{ times}}, 2^{-p}, 3^{-p}, \ldots, (d-R+1)^{-p}) \in \mathbb{R}^{d \times d}.$$

$$\text{(NoisyLR)} \quad \mathbf{A} = \operatorname{diag}(\underbrace{1, \ldots, 1}_{R \text{ times}}, 0, \ldots, 0) + \xi d^{-1} \mathbf{G}\mathbf{G}^* \in \mathbb{R}^{d \times d}.$$

$$\text{(ExpDecay)} \quad \mathbf{A} = \operatorname{diag}(\underbrace{1, \ldots, 1}_{R \text{ times}}, 10^{-q}, 10^{-2q}, \ldots, 10^{-(d-R)q}) \in \mathbb{R}^{d \times d}.$$

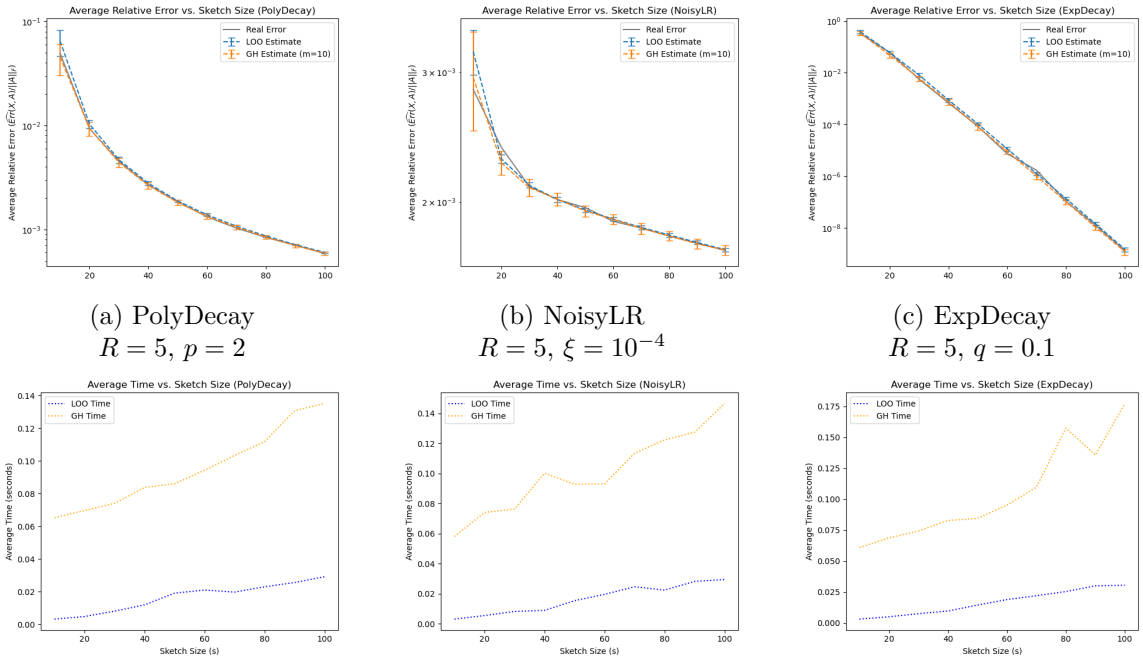
where $\xi, q \in \mathbb{R}$ are parameters, and $\mathbf{G} \in \mathbb{R}^{d \times d}$ is a standard Gaussian matrix. In this experiment, we set $d = 1000$.

Estimation Comparison Recall the conclusion in Section 2 (Methodology), compared to the GH method, LOO can obtain an **unbiased error estimation** with less time. In average, the average calculation time of the GH method is **5-15 times** that of the LOO method (Figure 2def). Moreover, the Jackknife estimator Jack(X) overestimates the standard deviation by a modest amount (Figure 3abc) and this is a **stable variance estimator**. However, the Bootstrap estimator Boot(X) performs well on *PolyDecay*, but underestimates the variance on *NoisyLR* and overestimates it on *ExpDecay* (Figure 3abc). More specifcly, the Jackknife estimator **maintains a consistent approximation** of the true standard deviation **across all matrix types**, with relative errors generally within

one order of magnitude (Ratio in Table 1). In contrast, the Bootstrap estimator **exhibits high variability** on *ExpDecay*, it overestimates the standard deviation by up to three orders of magnitude, while on *NoisyLR*, it underestimates by nearly half an order (Ratio in Table 1). These results highlight the **instability of Bootstrap under varying matrix structures**.

Matrix Type	**True Std.**	**Estimated Std.**		**Ratio (Estimate / True)**	
		Jackknife	Bootstrap	Jackknife	Bootstrap
PolyDecay ($R = 5$, $p = 2$)	1.16×10^{-3}	2.40×10^{-3}	2.05×10^{-3}	2.06	1.77
NoisyLR ($R = 5$, $\xi = 10^{-4}$)	2.13×10^{-3}	2.52×10^{-3}	$\mathbf{1.37 \times 10^{-3}}$	1.19	**0.64**
ExpDecay ($R = 5$, $q = 0.1$)	3.04×10^{-9}	2.28×10^{-8}	$\mathbf{2.19 \times 10^{-5}}$	7.50	**7210.00**

Table 1: Comparison of true and estimated std. and their ratios at sketch size $s = 100$



(a) PolyDecay $R = 5$, $p = 2$

(b) NoisyLR $R = 5$, $\xi = 10^{-4}$

(c) ExpDecay $R = 5$, $q = 0.1$

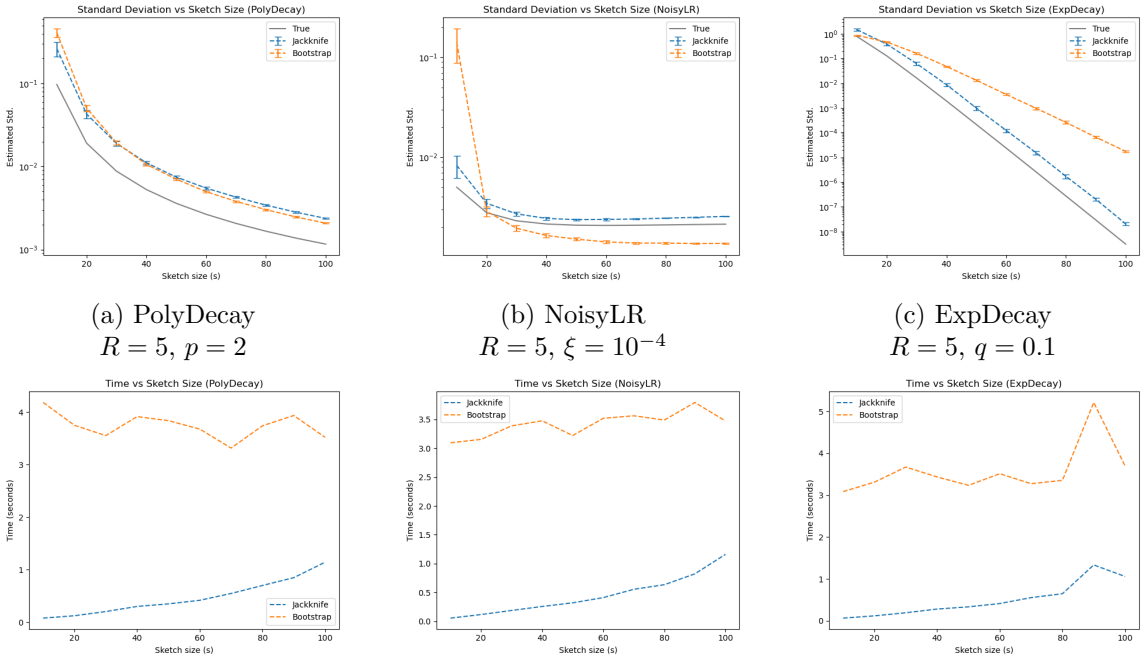
(d) PolyDecay $R = 5$, $p = 2$

(e) NoisyLR $R = 5$, $\xi = 10^{-4}$

(f) ExpDecay $R = 5$, $q = 0.1$

Figure 2: Leave-one-out and Girard-Hutchinson error estimator for randomized SVD

Noticed that we set $m = 10$ in GH method and we repeat the error estimation 100 times to compute the standard error for constructing confidence intervals and to calculate the average runtime.



(a) PolyDecay $R = 5$, $p = 2$

(b) NoisyLR $R = 5$, $\xi = 10^{-4}$

(c) ExpDecay $R = 5$, $q = 0.1$

(d) PolyDecay $R = 5$, $p = 2$

(e) NoisyLR $R = 5$, $\xi = 10^{-4}$

(f) ExpDecay $R = 5$, $q = 0.1$

Figure 3: Matrix Jackknife and Efron Bootstrap variance estimator for randomized SVD

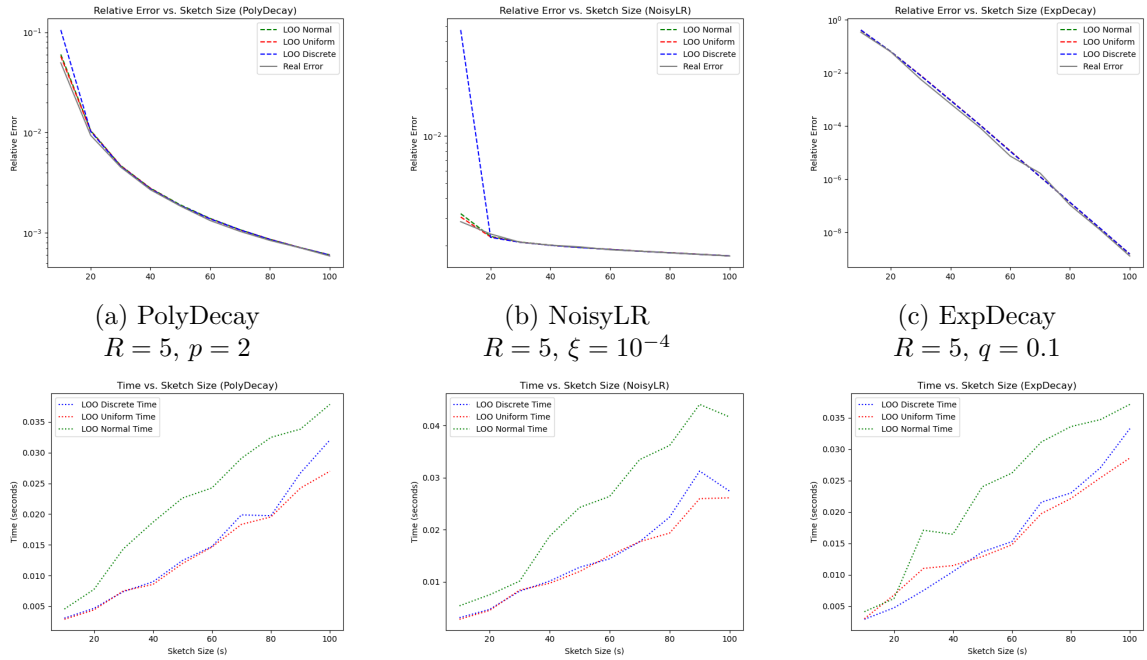
Noticed that the Bootstrap uses $B = 100$, and true std is estimated with 100 reps. And we repeat the variance estimation 100 times to compute the standard error for constructing confidence intervals and to calculate the average runtime.

Computational Efficiency Comparison We also compare the computational efficiency of the error estimators. **For error estimation**, when we set $m = 10$ in GH method, as we mentioned above, it requires approximately 5–15 times more computation time than the LOO estimator (Figure 2def). **For variance estimation**, in addition to providing a more stable estimate, the Jackknife method is also more computationally efficient. To ensure robustness, we set the number of bootstrap samples to $B = 100$. Under this setting, the Bootstrap method consistently requires 3–10 times more computation time than the Jackknife method (Figure 3def).

Different Random Sketching Matrices Comparison We have also made some new discoveries regarding the selection of different Random Sketching Matrix Ω. We conducted simulations on the choice of different Ω. From the earlier derivation, we know that all the theories suffices for the vectors ω_i to be independent and satisfy $\mathbb{E}[\omega_i \omega_i^T] = I$. Therefore,

we tested three types of distributions for generating ω_i: **Gaussian**, **Rademacher** (discrete ± 1), and **Uniform** distributions (Figure 4).

The experimental results show that, regardless of the sketch size s, using Gaussian random matrices incurs higher computational cost. As a result, for practical purposes such as error estimation, **using a uniformly distributed random matrix** Ω can achieve similar accuracy with lower runtime.



(a) PolyDecay $R = 5$, $p = 2$ (b) NoisyLR $R = 5$, $\xi = 10^{-4}$ (c) ExpDecay $R = 5$, $q = 0.1$

(d) PolyDecay $R = 5$, $p = 2$ (e) NoisyLR $R = 5$, $\xi = 10^{-4}$ (f) ExpDecay $R = 5$, $q = 0.1$

Figure 4: Different Ω in LOO error estimator for randomized SVD

Random Sketching Matrix $\Omega = \{\omega_{ij}\}_{d_2 \times s} \in \mathbb{R}^2$ *is generated from the following distribution:*

- **Gaussian:** Each entry $\omega_{ij} \sim \mathcal{N}(0, 1)$
- **Rademacher:** $\omega_{ij} = \pm 1$ with equal probability
- **Uniform:** $\omega_{ij} \sim \mathcal{U}(-\sqrt{3}, \sqrt{3})$ to ensure variance 1